



7 DE MAYO DE 2026

## ACTIVIDAD UT7 – ALUMNOS SIN FCT

JESÚS DANIEL MATEO GARCÍA

JESUS DANIEL MATEO – DAM1

BASES DE DATOS

I.E.S JULIO VERNE



# ACTIVIDAD UT7

## ALUMNOS SIN FCTs

### BBDD NOSQL

#### BLOQUE 1 (CE 7.a)

Responder razonadamente a las siguientes cuestiones:

1. **¿Qué es una BD NoSQL?**

El lenguaje NoSQL surgió como alternativa debido a la creciente demanda de consultas más rápidas ya que ofrece una escalabilidad rápida al no tratar con datos estructurados, es decir, a diferencia de la BD relacionales, estas no se unen por tablas por tanto no depende las unas de las otras, sino que albergan los datos dentro de una estructura de datos, como un documento JSON.

2. **¿Diferencias entre BD SQL y NoSQL?**

Básicamente se diferencia entre si porque la SQL son relacionales y las NoSQL no lo son, es decir, no están conectadas por tablas con clave primaria (id). Sus capacidades integran fragmentación y alta disponibilidad lo que facilita el escalado horizontal, a diferencia de la SQL que solo permite vertical, es decir, si quisieran ampliar el almacenamiento depende del Hardware, mientras que la NoSQL pueden dividirse en varios servidores.

3. **¿En qué casos usarías cada una?**

Principalmente, las BD SQL relacionales son una buena opción cuando se trabaja con datos relacionales, es decir, cuando un usuario actualiza un registro específico, cada instancia relacionada se actualiza automáticamente.

#### BLOQUE 2 (CE 7.b)

Responde razonadamente a las siguientes cuestiones:

1. **¿Cuáles son los tipos de BD NoSQL? Describelas brevemente con un párrafo y menciona 3 SGBD NoSQL conocidos por cada tipo.**

- **Almacén de valores clave.**

Este modelo de datos sin esquema se organiza en un diccionario de pares clave-valor. Se utiliza normalmente para guardar la información de la sesión del usuario.

- **Almacén de documentos.**

En este caso, se almacenan datos como documentos por lo general en formatos JSON, XML o BSON.

- **BD orientados a grafos.**

Este modelo usa estructuras de grafos para consultas semánticas y representa los datos como nodos, bordes y propiedades.

Ejemplo de BD orientado a grafos: FleckDB, InfinityGraph y OpenLink.

- **BD Tabulares.**  
Ofrecen una solución híbrida entre los DBMS tabulares y los de clave-valor. Un formato que permite como se nombran y formatean los datos en cada fila dentro de la misma tabla. Este tipo de BD No SQL acelera las consultas al guardar los datos en columnas.
- **BD Orientadas a objetos.**  
Combina las BD con los lenguajes de programación orientados a objetos. Los datos se representan en objetos.  
Ejemplo de BD orientadas a objetos: Db40, GemStone, Realm.

2. ¿Qué tipo de BD es MongoDB? ¿Y Redis? Completa la siguiente tabla brevemente:

### **MongoDB**

Es una BD NoSQL, orientada a documentos, de código abierto y distribuida. Utiliza un modelo de documentos flexibles (similares a JSON/BSON) que permite usar colecciones o documentos en vez de tablas y columnas.

### **Redis**

Es una BD NoSQL en memoria que almacena datos muy rápido. Especialmente utilizada en cache, sesiones, colas y datos temporales.

| Criterio              | MongoDB                                    | Redis                                    |
|-----------------------|--------------------------------------------|------------------------------------------|
| Modelo de datos       | NoSQL                                      | NoSQL                                    |
| Estructura            | Documento BSON/JSON                        | Valores clave-valor                      |
| Persistencia de datos | Persiste en disco                          | En memoria RAM                           |
| Velocidad             | Alta                                       | Muy alta                                 |
| Consultas             | Consultas avanzadas                        | Consultas simples                        |
| Datos complejos       | Muy útil para datos complejos              | Limitado                                 |
| Escalabilidad         | Muy buena<br>(Particionamiento horizontal) | Muy buenas, pero más costosa al usar RAM |
| Replicación           | Si admite                                  | Si admite                                |
| Relaciones            | No tiene relaciones (NoSQL)                | No tiene relaciones                      |
| Uso principal         | BD de una app                              | Cache, sesiones, datos temporales, etc.  |
| Casos de uso real     | App's web, APIs, etc.                      | Chats, notificaciones, etc.              |
| IDE                   | Mongo Compass                              | RedisInsight                             |

3. ¿A que nos referimos con escalabilidad y replicación de un BD?

Consiste en el proceso para adaptarlas para manejar varios datos y usuarios de manera eficiente. Esto se logra mejorando el Hardware existente (escalado vertical) o agregando mas servidores (escalado horizontal).

Técnicas como el particionamiento (sharding) y la replicación son claves.

- **REPLICACION.** Es el proceso mediante el cual los datos que residen un servidor físico/virtual o instancia en la nube se replican o copian continuamente en un servidor secundario o instancia en la nube (instancia de respaldo).

Su principal objetivo es respaldar la alta disponibilidad, la copia de seguridad y/o la recuperación ante desastres.

- **PARTICIONAMIENTO.** Es una técnica para dividir un conjunto de datos grandes en fragmentos más pequeños, distribuyendo estos fragmentos en diferentes maquinas o nodos de BD para distribuir la carga de tráfico. No todas las BD admiten 'Sharding'.

### BLOQUE 3 (7.c / 7.d)

Responde razonadamente a las siguientes cuestiones:

1. Explica los conceptos relativos a MongoDB: colección, documento, campo, \_id.
  - **Colección.**  
Es un conjunto de documentos, lo equivalente a una tabla en SQL.
  - **Documento.**  
Es una unidad de información en MongoDB, lo equivalente a una fila en SQL. Un documento guarda los datos en formato JSON / BSON.
  - **Campo.**  
Es cada propiedad dentro de un documento, es decir, edad: 20, edad sería el campo.
  - **\_id.**  
Es el identificador único, es obligatorio, no se puede repetir y si no lo pones, MongoDB lo genera automáticamente.

2. Tenemos el siguiente código:

```
[
  {
    "id": 1,
    "nombre": "Camiseta Deportiva",
    "categoria": "Ropa",
    "precio": 19.99,
    "stock": 120,
    "tallas": ["S", "M", "L", "XL"],
    "detalles": {
      "material": "Algodón",
      "color": "Azul"
    }
  },
  {
    "id": 2,
    "nombre": "Auriculares Bluetooth",
    "categoria": "Electrónica",
    "precio": 49.90,
    "esOferta": true,
    "especificaciones": {
      "conectividad": "Bluetooth 5.0",
      "autonomia_horas": 12
    }
  },
  {
    "id": 3,
    "nombre": "Libro 'MongoDB Avanzado'",
    "categoria": "Libros",
    "precio": 35.00,
    "stock": 50,
    "autores": ["Jorge Carmena", "María López"],
    "idioma": "Español",
    "temas": ["Bases de datos", "NoSQL", "MongoDB"]
  }
]
```

Rodeo en rojo las colecciones, en verde los documentos y subraya en amarillo los campos. ¿Qué tipos identificas en ese código?

- Podemos apreciar en el código diferentes tipos de variables con int (stock: 50), String (categoría: electrónica), boolean (esOferta:true), Array (temas: [BD, NoSQL, MongoDB] y objetos (detalles o especificaciones).

3. Tenemos dos modelos de datos alternativos para una librería. Los datos tienen exactamente los mismos datos, pero distribuidos de diferente manera:

| A                                                                                                                                                                                                                                                                                                                                                      | B                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>LIBROS:</b><br><pre>[   {     "_id": 1,     "titulo": "El Principito",     "autor": {       "nombre": "Antoine de Saint-Exupéry",       "nacionalidad": "francesa"     }   },   {     "_id": 2,     "titulo": "Cien Años de Soledad",     "autor": {       "nombre": "Gabriel García Márquez",       "nacionalidad": "colombiana"     }   } ]</pre> | <b>AUTORES:</b><br><pre>[   {     "_id": 101,     "nombre": "Antoine de Saint-Exupéry",     "nacionalidad": "francesa"   },   {     "_id": 102,     "nombre": "Gabriel García Márquez",     "nacionalidad": "colombiana"   } ]</pre> <b>LIBROS:</b><br><pre>[   {     "_id": 1,     "titulo": "El Principito",     "autor_id": 101   },   {     "_id": 2,     "titulo": "Cien Años de Soledad",     "autor_id": 102   } ]</pre> |

¿Cuál es el modelo referenciado y cual el 'embebido'? Describe el por qué y ventajas y desventajas de cada uno.

#### **Modelo A**

Es el embebido porque la información del autor esta dentro del libro y no separada en otra colección.

#### **Modelo B**

Es el referenciado porque autores y libros se encuentran en colecciones independientes y el libro guarda una referencia del autor mediante 'autor\_id'.

4. Explica los conceptos relativos a Redis: Clave, valor.

- **Clave.** Consiste básicamente en el valor del dato, debe ser único y se utiliza para poder recuperar el 'Valor' cuando sea necesario.
- **Valor.** Es la información que se guarda dentro de la clave, puede ser un número, un grupo de caracteres, una lista, etc.

5. Explica los conceptos relativos a Redis: SET, HSET, LPUSH.

- **SET.** Permite guardar un valor simple asociado a una clave.  
Ejemplo.

**SET** nombre "Daniel"

- **HSET.** Permite guardar varios campos dentro de una misma clave como un objeto en JSON.  
Ejemplo.

**HSET** usuario1 nombre "Daniel"

**HSET** usuario1 edad 28

- **LPUSH.** Sirve para insertar elementos en una lista.  
Ejemplo.

**LPUSH** carrito "producto1"

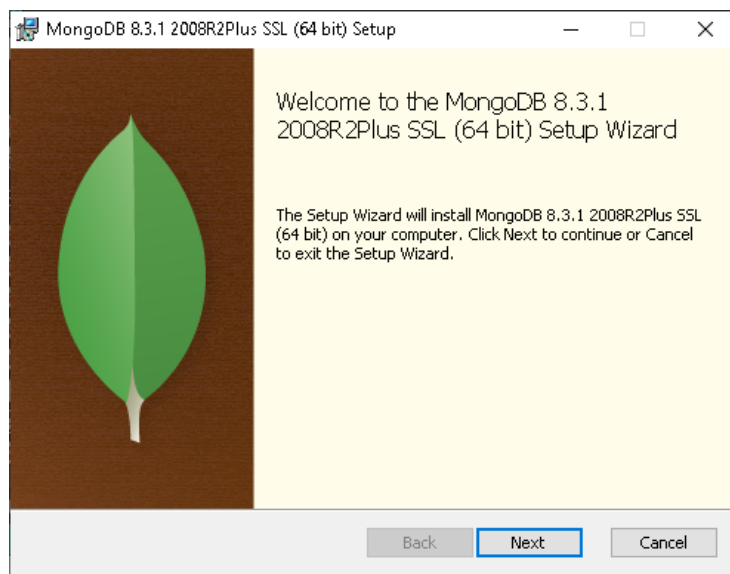
**LPUSH** carrito "producto2"

6. Supón que se está desarrollando una app de comercio electrónico. ¿Cuál de los siguientes datos se almacenarían en MongoDB y cuales en Redis?

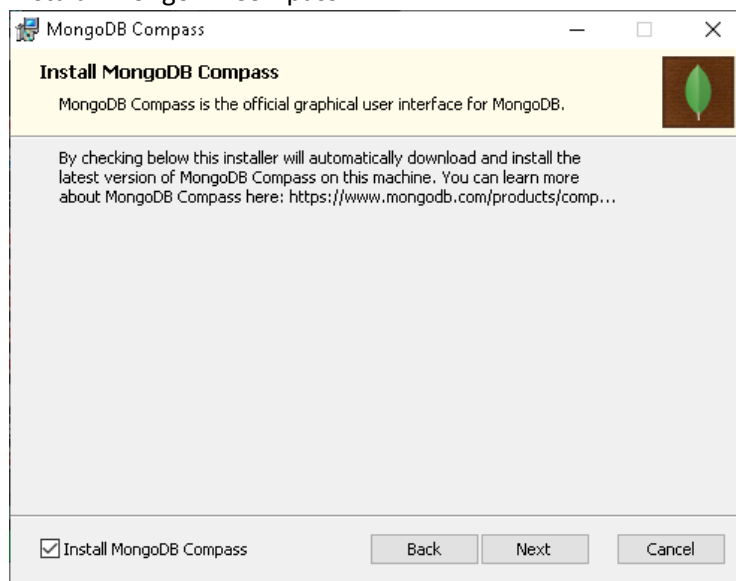
| Dato                                    | BD (MongoDB/Redis) | Por qué                              |
|-----------------------------------------|--------------------|--------------------------------------|
| Usuarios                                | MongoDB            | Datos más importantes y persistentes |
| Pedidos                                 | MongoDB            | Datos más importantes y persistentes |
| Carrito de la compra                    | Redis              | Datos temporales                     |
| Contadores de visitas sobre un producto | Redis              | Datos temporales                     |
| Productos                               | MongoDB            | Datos más importantes y persistentes |
| Últimos productos                       | Redis              | Datos temporales                     |

#### BLOQUE 4 (7.e)

- Seguir los pasos siguientes y hacer capturas de pantalla de cada paso:
  - Instalar el servidor MongoDB Community Edition 7.0.



b) Instalar MongoDB Compass.



c) Arrancar el servidor (si no lo está).

|                                                                                                        |       |              |                 |    |           |               |
|--------------------------------------------------------------------------------------------------------|-------|--------------|-----------------|----|-----------|---------------|
|  mongod.exe         | 4020  | En ejecución | Servicio de red | 00 | 16.600 K  | No permiti... |
|  MongoDBCompass.exe | 14380 | En ejecución | DAW2            | 01 | 79.636 K  | Deshabilit... |
|  MongoDBCompass.exe | 9516  | En ejecución | DAW2            | 00 | 4.400 K   | Deshabilit... |
|  MongoDBCompass.exe | 14296 | En ejecución | DAW2            | 00 | 59.968 K  | Deshabilit... |
|  MongoDBCompass.exe | 16524 | En ejecución | DAW2            | 00 | 7.368 K   | Deshabilit... |
|  MongoDBCompass.exe | 540   | En ejecución | DAW2            | 00 | 127.656 K | Deshabilit... |

## New Connection

Manage your connection settings

URI ⓘ

Edit Connection String 

mongodb://localhost:27017/

Name

Color

No Color ▾

☐ **Favorite this connection**  
 Favoriting a connection will pin it to the top of your list of connections

> Advanced Connection Options

Cancel

Save

Connect

Save & Connect

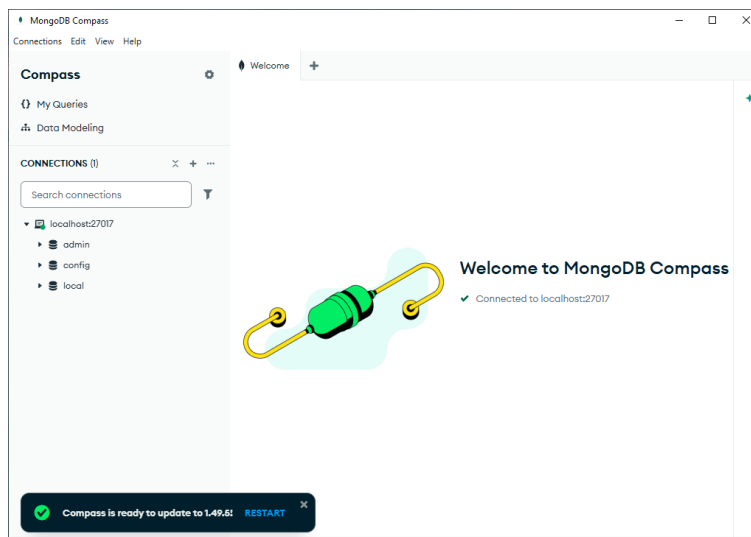
**How do I find my connection string in Atlas?**

If you have an Atlas cluster, go to the Cluster view. Click the 'Connect' button for the cluster to which you wish to connect.  
[See example](#)

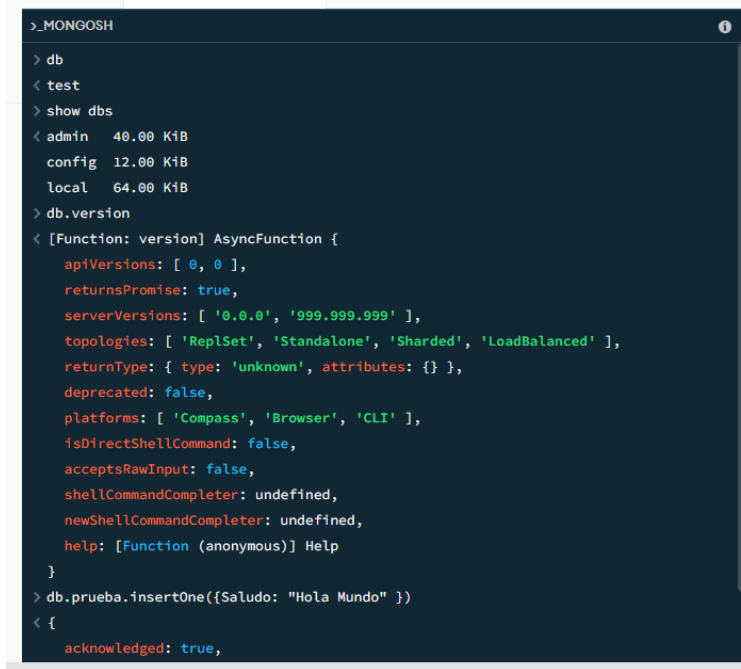
**How do I format my connection string?**

[See example](#)

d) Conectarse al servidor desde Compass.



- e) Abrir una pestaña MongoDB Shell (mongosh) en Compass.



```
> _MONGOSH
> db
< test
> show dbs
< admin   40.00 KiB
  config  12.00 KiB
  local   64.00 KiB
> db.version
< [Function: version] AsyncFunction {
  apiVersions: [ 0, 0 ],
  returnsPromise: true,
  serverVersions: [ '0.0.0', '999.999.999' ],
  topologies: [ 'ReplSet', 'Standalone', 'Sharded', 'LoadBalanced' ],
  returnType: { type: 'unknown', attributes: {} },
  deprecated: false,
  platforms: [ 'Compass', 'Browser', 'CLI' ],
  isDirectShellCommand: false,
  acceptsRawInput: false,
  shellCommandCompleter: undefined,
  newShellCommandCompleter: undefined,
  help: [Function (anonymous)] Help
}
> db.prueba.insertOne({Saludo: "Hola Mundo" })
< {
  acknowledged: true,
```

2. Describir los siguientes conceptos de MongoDB:

a) **MongoDB.**

Es la interfaz gráfica de MongoDB Edition 7.0.

b) **Compass.**

Es la interfaz gráfica de MongoDB, es el punto visual.

c) **Mongosh.**

También conocido como 'Mongo Shell', consiste en la consola de comandos.

d) **Colección.**

Es un conjunto de documentos, lo equivalente a una tabla en SQL.

e) **Documento.**

Es una unidad de información en MongoDB, lo equivalente a una fila en SQL. Un documento guarda los datos en formato JSON / BSON.

f) **Campo.**

Es cada propiedad dentro de un documento, es decir, edad: 20, edad sería el campo.

g) **\_id.**

Es el identificador único, es obligatorio, no se puede repetir y si no lo pones, MongoDB lo genera automáticamente.

3. Describir los siguientes comandos de mongosh:

| COMANDO                                              | DESCRIPCION                                                           |
|------------------------------------------------------|-----------------------------------------------------------------------|
| show dbs                                             | Muestra todas las BD disponibles en MongoDB                           |
| show collections                                     | Muestras todas las colecciones                                        |
| use <BD>                                             | Cambia la BD indicado. Si no existe, la crea al insertar datos.       |
| db.<COLECCIÓN>.insertMany(<ARRAY_JSON>)              | Inserta varios documentos en una colección                            |
| db.<COLECCIÓN>.insertOne(<OBJETO_JSON>)              | Inserta un único documento en una colección                           |
| db.<COLECCIÓN>.find({<FILTROS>}).pretty()            | Busca documentos que cumplan con el filtro y los muestra formateados. |
| db.<COLECCIÓN>.updateOne({<FILTROS>}, {<OPERADOR>})  | Actualiza el primer documento que coincida con el filtro              |
| db.<COLECCIÓN>.updateMany({<FILTROS>}, {<OPERADOR>}) | Actualiza todos los documentos que coincidan con el filtro            |
| db.<COLECCIÓN>.deleteOne({<FILTROS>})                | Elimina el primer documento que coincida con el filtro                |
| db.<COLECCIÓN>.deleteMany({<FILTROS>})               | Elimina todos los documentos que coincidan con el filtro              |

4. Describir los siguientes operadores de consultas (find) de MongoDB:

| OPERADOR | DESCRIPCION                                                   |
|----------|---------------------------------------------------------------|
| \$eq     | Igual a un valor                                              |
| \$ne     | Distinto a un valor                                           |
| \$gt     | Mayor que                                                     |
| \$gte    | Mayor o igual que                                             |
| \$lt     | Menor que                                                     |
| \$lte    | Menor o igual que                                             |
| \$in     | Coincide con cualquiera de los valores de una lista           |
| \$nin    | No coincide con ninguna de los valores de la lista            |
| \$or     | Devuelve documentos que cumplan al menos una condición        |
| \$and    | Devuelve documentos que cumplan todas las condiciones         |
| \$not    | Niega una condición                                           |
| \$nor    | Devuelve documentos que no cumplan ninguna de las condiciones |
| \$exists | Comprueba si un campo existe o no                             |
| \$type   | Filtra por un tipo de dato de un campo                        |
| \$regex  | Busca valores que coincidan con una expresión regex           |

5. Describe los siguientes operadores de actualización (update) de MongoDB:

| OPERADOR      | DESCRIPCION                                                                                   |
|---------------|-----------------------------------------------------------------------------------------------|
| \$set         | Reemplaza el valor de un campo con el valor especificado. Si el campo no existe, lo crea.     |
| \$unset       | Elimina un campo específico de un documento                                                   |
| \$inc         | Incrementa un valor de un campo numérico en la cantidad indicada                              |
| \$mul         | Multiplica el valor de un campo por un número determinado                                     |
| \$rename      | Cambia el nombre de un campo existente por uno nuevo                                          |
| \$min         | Actualiza el valor de un campo solo si el valor especificado es menor que el valor actual     |
| \$max         | Actualiza el valor de un campo solo si el valor especificado es mayor que el valor actual     |
| \$currentDate | Establece el valor de un campo a la fecha y hora actuales (ya sea como tipo Date o TimeStamp) |
| \$push        | Agrega un elemento al final de un array                                                       |
| \$addToSet    | Agrega un elemento a un array solo si el elemento no existe ya, evitando duplicados           |
| \$pop         | Elimina el primer (-1) o el último (1) elemento de un array                                   |
| \$pull        | Elimina de un array todos los elementos que coincidan con una condición o valor específico.   |
| \$pullAll     | Elimina de un array todas las ocurrencias de los elementos específicos en una lista           |

6. Se proporcionan los siguientes ficheros JSON: películas.json, usuarios.json y visualizaciones.json. Se pide resolver los siguientes apartados con mongosh:

- Crear una base de datos llamada “streaming” en el servidor de MongoDB.
- Insertar todos los documentos en sus respectivas colecciones.
- Realizar las siguientes consultas:
  - Todas las películas.

```
> MONGOSH
> use streaming;
< switched to db streaming
> db.películas.find()
< {
  _id: 103,
  título: 'Toy Story',
  genero: [
    'Animación',
    'Comedia'
  ],
  duracion_min: 81,
  'año': 1995,
  valoracion_medio: 8.3
}
{
  _id: 102,
  título: 'El Padrino',
  genero: [
    'Drama',
    'Crimen'
  ],
  duracion_min: 175,
  'año': 1972,
  valoracion_medio: 9.2
}
```

- Todas las películas de género 'Drama'.

```
> db.peliculas.find({ genero: "Drama" })
< {
  _id: 102,
  titulo: 'El Padrino',
  genero: [
    'Drama',
    'Crimen'
  ],
  duracion_min: 175,
  'año': 1972,
  valoracion_media: 9.2
}
```

- Todas las películas que duren más de 1h y 30min y que tengas una valoración media igual o superior a 8.

```
> _MONGOSH
> db.peliculas.find({
  duracion_min: { $gt: 90 },
  valoracion_media: { $gte: 8 }
})
< {
  _id: 102,
  titulo: 'El Padrino',
  genero: [
    'Drama',
    'Crimen'
  ],
  duracion_min: 175,
  'año': 1972,
  valoracion_media: 9.2
}
{
  _id: 101,
  titulo: 'Origen',
  genero: [
    'Ciencia ficción',
    'Acción'
  ],
  duracion_min: 148,
  'año': 2010,
  valoracion_media: 8.8
}
```

- Todas visualizaciones del usuario de ID=1.

```
> db.visualizaciones.find({ usuario_id: 1 })
< {
  _id: 1001,
  usuario_id: 1,
  perfil: 'Ana',
  fecha: '2026-03-20',
  pelicula: {
    id: 101,
    titulo: 'Origen',
    duracion_min: 148
  },
  progreso: 100,
  valoracion: 9
}
{
  _id: 1002,
  usuario_id: 1,
  perfil: 'Niños',
  fecha: '2026-03-21',
  pelicula: {
    id: 103,
    titulo: 'Toy Story',
    duracion_min: 81
  },
  progreso: 100,
  valoracion: 9
}
```

- d) Realizar las siguientes actualizaciones de datos:

- El usuario de ID=2 acaba de suscribirse al plan 'premium'.

```
> db.usuarios.updateOne(
  { _id: 2 },
  { $set: { plan: "premium" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
> db.usuarios.find()
< {
  _id: 1,
  nombre: 'Ana García',
  email: 'ana@email.com',
  plan: 'premium',
  fecha_registro: '2025-11-10',
  perfiles: [
    {
      nombre: 'Ana',
      edad: 30
    },
    {
      nombre: 'Niños',
      edad: 8
    }
  ]
}
{
  _id: 2,
  nombre: 'Luis Pérez',
  email: 'luis@email.com',
  plan: 'premium',
  fecha_registro: '2026-01-05',
  perfiles: [
    {
      nombre: 'Luis',
      edad: 35
    },
    {
      nombre: 'Ana',
      edad: 33
    }
  ]
}
```

Plan  
premium

- El usuario de ID=2 acaba de agregar un nuevo perfil de nombre 'Ana' y de edad '33'.

```
> db.usuarios.updateOne(
  { _id: 2 },
  {
    $push: {
      perfiles: { nombre: "Ana", edad: 33 }
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
> db.usuarios.find()
< {
  _id: 2,
  nombre: 'Luis Pérez',
  email: 'luis@email.com',
  plan: 'premium',
  fecha_registro: '2026-01-05',
  perfiles: [
    {
      nombre: 'Luis',
      edad: 35
    },
    {
      nombre: 'Ana',
      edad: 33
    }
  ]
}
```

Nuevo  
perfil

- El usuario de ID=2 ha terminado de ver la película de ID=102 al 100% y la ha valorado con 10.

```
> db.visualizaciones.updateOne(
  {
    usuario_id: 2,
    "película.id": 102
  },
  {
    $set: {
      progreso: 100,
      valoracion: 10
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
{
  _id: 1003,
  usuario_id: 2,
  perfil: 'Luis',
  fecha: '2026-03-22',
  película: {
    id: 102,
    titulo: 'El Padrino',
    duracion_min: 175
  },
  progreso: 100,
  valoracion: 10
}
```

Progreso  
100%

Valoración  
10

e) Realizar los siguientes borrados de documentos:

- El usuario de ID=2.

```
> db.usuarios.deleteOne({ _id: 2 })
< {
  acknowledged: true,
  deletedCount: 1
}
> db.usuarios.find()
< {
  _id: 1,
  nombre: 'Ana García',
  email: 'ana@email.com',
  plan: 'premium',
  fecha_registro: '2025-11-10',
  perfiles: [
    {
      nombre: 'Ana',
      edad: 30
    },
    {
      nombre: 'Niños',
      edad: 8
    }
  ]
}
```

- Las películas de genero de 'Comedia' o 'Drama'.

```
> db.películas.deleteMany({
  genero: { $in: ["Comedia", "Drama"] }
})
< {
  acknowledged: true,
  deletedCount: 2
}
> db.películas.find()
< {
  _id: 101,
  titulo: 'Origen',
  genero: [
    'Ciencia ficción',
    'Acción'
  ],
  duracion_min: 148,
  'año': 2010,
  valoracion_media: 8.8
}
```